COURSE PROJECT REPORT

Project Title: Game Save Corruption and Recovery Engine

Course: Advanced Data Structures

Submitted by:

Aviral Shirgupe | 612415169

Sheel Gattani | 612415163

Divyam Shah | 612415160

Ved Bajaj | 612415191

Under the guidance of: Prof. Savita Adhav

Computer Science and Engineering

COEP TECHNOLOGICAL UNIVERSITY

Academic Year: 2026-2027

CERTIFICATE

This is to certify that the course project entitled “Game Save Corruption and Recovery Engine” has been carried out by Aviral Shirgupe, Sheel Gattani, Divyam Shah and Ved Bajaj of Computer Science and Engineering under my guidance during the academic year 2026-2027. The work is submitted in partial fulfilment of the requirements of the course.

Project Guide: _____ Head of Department: _____

Date: _____ Place: _____

DECLARATION

We hereby declare that the project work entitled “Game Save Corruption and Recovery Engine” is our own work carried out under the guidance of Prof. Savita Adhav. Sources of information used in this report have been acknowledged appropriately.

Student signature(s): _____

Student signature(s): _____

Student signature(s): _____

Student signature(s): _____

ACKNOWLEDGEMENT

We thank our project guide Prof. Savita Adhav, for the guidance and support given throughout this project. We also thank the Department of Computer Science & Engineering, COEP Technological University, for giving us this opportunity, and each other as team members for working together on it.

ABSTRACT

Game save files can get corrupted when the game crashes while saving, when the disk is faulty, when the game has a bug, or when someone edits the file to cheat. This project is a Game Save Corruption and Recovery Engine written in C. It stores the player's progress as checkpoints, checks every saved state for corruption, and brings the player back to the best valid state.

Each part uses one data structure: an AVL tree stores the checkpoints, a trie stores the list of valid items, a graph stores the order of quests, and a max-heap picks the best checkpoint to recover. Corruption is found in two ways: a checksum catches damaged bytes, and rule checks catch impossible values. A scripted demo shows every feature, and all tests passed.

CHAPTER 1: INTRODUCTION

1.1 Background

Video games save the player's progress (health, money, items and completed quests) so the player can continue later. Many games also keep checkpoints, which are saved copies of the game at different moments. If a save gets damaged, the player can lose progress or the game can crash. This project builds a small engine that protects game saves, as part of the Advanced Data Structures course.

1.2 Problem Statement

A save can go wrong in two ways:

- **Damaged bytes:** some data changes because of a power cut or a disk error.
- **Impossible values:** the data is readable but makes no sense, for example 300 HP when the maximum is 100, an item that does not exist, or the last quest finished before the first ones.

A checksum can catch the first problem but not the second. We need a system that catches both and can bring the player back to a good save.

1.3 Aim and Objectives

Aim: To build a system that finds corrupted game saves and recovers the best valid one, using a suitable data structure for each part.

Objectives:

- Store checkpoints in time order using an AVL tree.
- Check items against a list of valid items using a trie.
- Check quest order using a graph.
- Find corruption and pick the best checkpoint to recover using a max-heap.

1.4 Scope of the Project

Included: a simple game (health, gold, items, quests), checkpoints, save and load, item search, corruption testing, detection, rollback and recovery, a menu and a demo.

Not included: graphics, online play, and keeping checkpoints after the program closes.

1.5 Significance / Motivation

Keeping data safe from corruption matters in games, databases and file systems. This project shows the idea in a small and simple way, and it shows how different data structures can work together to solve one real problem.

CHAPTER 2: LITERATURE REVIEW / EXISTING SYSTEM

2.1 Review of Related Work

- **Checksums:** used in ZIP files and network data to find damaged bytes.
- **Database rules:** databases reject values that break rules, such as values outside a valid range.
- **Game backups:** many games keep backup saves and check for edited saves.

2.2 Existing System and Limitations

- Many simple games keep only one save file. If it is damaged, all progress is lost.
- A checksum alone misses impossible values written by a buggy game.
- Going back to the latest save may put the player in a bad spot, for example with very low health.

2.3 Gap Identified

Our project uses both a checksum and rule checks, keeps a history of checkpoints, and picks the best valid checkpoint instead of just the latest one.

CHAPTER 3: REQUIREMENTS AND METHODOLOGY

3.1 Proposed System

The whole game at one moment is stored in one structure called GameState. It also stores a checksum, which is updated after every normal change. The project has four parts, one per member:

- **Checkpoints:** AVL tree.
- **Items:** Trie.
- **Quests:** Graph.
- **Detection and recovery:** Max-Heap.

3.2 Functional Requirements

- Let the user play: explore, pick up items and complete quests.
- Create, list and roll back checkpoints.
- Save and load the game.
- Search items by their first letters.
- Add corruption on purpose for testing, then detect it.
- Compare two saves and recover the best valid one.

3.3 Non-functional Requirements

- Never load or save a corrupted state.
- Fast operations (O(log n) or better for the main tasks).
- Easy menu with simple inputs.
- Demo gives the same output every time.

3.4 Tools and Technologies

- **Language:** C
- **Compiler:** GCC
- **System:** Linux or Windows (MinGW), run in a terminal

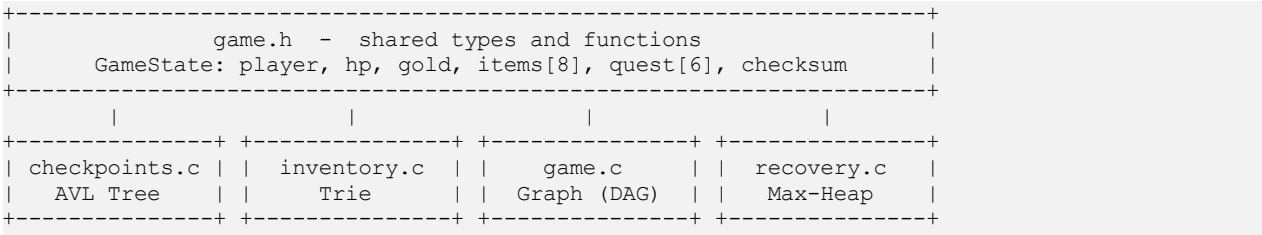
3.5 Methodology / Workflow

- First, the team agreed on one shared header file, game.h.
- Then each member built their own part.
- Next, all parts were joined into one program.
- Finally, a demo was written to test every feature.

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture / Block Diagram

All four parts share one header file, game.h, which defines GameState and the functions each part offers to the others.



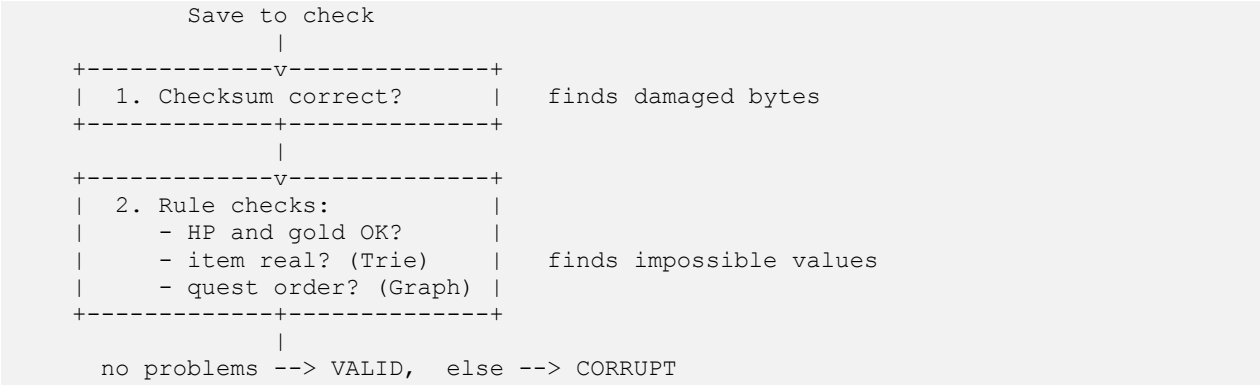
The recovery part uses the trie to check items and the graph to check quests.

4.2 Flowchart / UML / Circuit / Database Design

Table 1: Fields of the GameState structure

Field	Meaning
player	Player name
hp	Health (0 to 100)
gold	Money
items	Up to 8 items, each with a name and quantity
quest	Status of 6 quests: locked, active or done
checksum	A number calculated from all the other fields

How a save is checked:



4.3 Module Description

Table 2: Modules and data structures

File	Data structure	What it does
checkpoints.c	AVL tree	Stores checkpoints by time; rollback
inventory.c	Trie	Valid items, inventory, item search, compare saves
game.c	Graph	Quest order, gameplay, menu, demo
recovery.c	Max-Heap	Checksum, checks, save/load, recovery

AVL tree: a binary search tree that keeps itself balanced, so finding a checkpoint stays fast even though checkpoints are always added in time order.

Trie: a tree that stores words letter by letter. It stores the 8 valid item names and how many of each can be stacked. It can quickly say if an item is real, and list all items starting with some letters.

Graph: the 6 quests and which ones must be done first. It checks that there are no loops and gives a correct order to play the quests.

Max-Heap: gives each valid checkpoint a score and always keeps the highest score at the top, so the best checkpoint can be picked quickly.

4.4 Algorithm / Pseudocode

Trie lookup: follow the item name letter by letter from the root. If a letter is missing, or the last node is not marked as the end of a word, the item does not exist.

AVL insert: add the checkpoint like a normal binary search tree, then rotate nodes if one side becomes too tall.

Rollback: find the latest checkpoint before the given time. If it is corrupted, try the one before it, until a valid one is found.

Quest order (Kahn's algorithm): start with quests that need nothing, and keep adding quests whose required quests are already done.

Best recovery: put every valid checkpoint into the max-heap with a score (more quests and gold is better, very low health is worse), then take the top one.

Table 3: Time complexity

Operation	Time
AVL insert / search	O(log n)
Trie lookup	O(L), L = length of the name
Graph quest order / loop check	O(V + E)
Heap insert / remove top	O(log n)

CHAPTER 5: IMPLEMENTATION

5.1 Implementation Details

The project is written in C in five files: game.h, checkpoints.c, inventory.c, game.c and recovery.c. Every normal change to the game updates the checksum. The program can run as a menu, or as an automatic demo with the --demo option.

5.2 Key Code / Hardware Details

Build and run:

```
gcc -Wall -Wextra checkpoints.c inventory.c game.c recovery.c -o game_recovery
./game_recovery            (menu)
./game_recovery --demo     (demo)
```

Example – checking if an item exists (trie):

```
int trie_get(TrieNode *root, const char *key, int *value)
{
    TrieNode *n = walk(root, key);
    if (!n || !n->end) return 0;      /* not a valid item */
    if (value) *value = n->value;     /* max stack size  */
    return 1;
}
```

5.3 Individual Contributions

Table 4: Individual contributions

Member	Part	Work done
Ved	AVL tree	Checkpoints, AVL tree, rollback
Aviral	Trie	Item list, trie, inventory, item search, compare saves, helper functions
Divyam	Graph	Quest graph, gameplay, menu, demo, main()
Sheel	Max-Heap	Checksum, checks, save/load, corruption testing, recovery

CHAPTER 6: TESTING, RESULTS AND DISCUSSION

6.1 Test Plan and Test Cases

Testing is done with the demo (./game_recovery --demo). It runs every feature one by one and gives the same output every time. It will also be shown live.

Table 5: Test cases (demo)

Test ID	Test Case / Input	Expected Result	Actual Result / Status
TC-01	Play and create 6 checkpoints	Checkpoints stored in time order	Pass
TC-02	Save the game	File saved	Pass
TC-03	Search items (trie)	Items listed in order	Pass
TC-04	Show quest order (graph)	No loops; correct order	Pass
TC-05	Damage the save file, then load it	Load refused	Pass
TC-06	Corrupt 3 checkpoints	Corruption added	Pass
TC-07	Detect corruption	All 3 found with reasons	Pass
TC-08	Compare two saves	Differences shown	Pass
TC-09	Rollback	Latest valid checkpoint restored	Pass
TC-10	Recover best (heap)	Best checkpoint restored	Pass
TC-11	Try to save a corrupted game	Save refused	Pass

6.2 Results / Screenshots

Results from the demo:

Table 6: Checkpoint status after corruption

Checkpoint	Status	Reason
t=37	Valid	–
t=62	Corrupt	Quest done before its required quest (graph)
t=82	Corrupt	9999 potions and a fake item (trie)
t=102	Valid	–
t=103	Valid	–
t=118	Corrupt	Damaged bytes (checksum)

Table 7: Recovery ranking

Rank	Checkpoint	Health	Score
1	t=102	58	4399
2	t=103	10 (low)	3399
3	t=37	88	1153

Rollback restored t=103, the latest valid checkpoint, but the player had only 10 health there. Recovery picked t=102 instead, which has the same progress and more health.

```
aviral@aviral-Nitro-ANV15-51:~/Downloads/Game_Save_Recovery_Engine /Game Save Recovery Engine$ ./game_recovery --demo

================================================================
  STEP 1 - NEW GAME, PLAY, CREATE CHECKPOINTS
================================================================
    Aria | HP 100/100 | Gold 50
    Inventory: health_potion x2
    Quests done:
  Explored: +33 gold, -12 HP
  Quest complete: Wake in Riverwood (+50 gold)
  >> New quest unlocked: Clear the Goblin Cave
  >> New quest unlocked: Find the Old Sword
  [+] Checkpoint t=37 "Left the village" stored  (AVL: 1 nodes, height 1)
  Quest complete: Find the Old Sword (+50 gold)
  >> New quest unlocked: Forge Dragonbane
  Explored: +27 gold, -17 HP
  [+] Checkpoint t=62 "Found the sword" stored  (AVL: 2 nodes, height 2)
  Quest complete: Clear the Goblin Cave (+50 gold)
  >> New quest unlocked: Consult the Oracle
  Picked up 2 x torch
  [+] Checkpoint t=82 "Cleared the cave" stored  (AVL: 3 nodes, height 2)
  Quest complete: Consult the Oracle (+50 gold)
  Explored: +49 gold, -13 HP
  [+] Checkpoint t=102 "Met the oracle" stored  (AVL: 4 nodes, height 3)
  !! A monster attack leaves Aria at 10/100 HP
  [+] Checkpoint t=103 "Hurt by a monster" stored  (AVL: 5 nodes, height 3)
  Quest complete: Forge Dragonbane (+50 gold)
  >> New quest unlocked: Slay the Dragon
  [+] Checkpoint t=118 "Forged Dragonbane" stored  (AVL: 6 nodes, height 3)

  Checkpoints (AVL in-order = time order):
    t=37    Left the village         HP   88  gold 133   quests 1/6  ok
```

```
  [+] Checkpoint t=118 "Forged Dragonbane" stored  (AVL: 6 nodes, height 3)

  Checkpoints (AVL in-order = time order):
    t=37    Left the village         HP   88  gold 133   quests 1/6  ok
    t=62    Found the sword          HP   71  gold 210   quests 2/6  ok
    t=82    Cleared the cave         HP   71  gold 260   quests 3/6  ok
    t=102   Met the oracle           HP   58  gold 359   quests 4/6  ok
    t=103   Hurt by a monster        HP   10  gold 359   quests 4/6  ok
    t=118   Forged Dragonbane        HP   10  gold 409   quests 5/6  ok
  6 checkpoints, tree height 3
  Tree structure:
                  +-- t=118
            +-- t=103
    ROOT t=102
                  +-- t=82
            +-- t=62
                  +-- t=37

================================================================
  STEP 2 - SAVE GAME
================================================================
  Game saved to save.sav (checksum 85306CAB)

================================================================
  STEP 3 - TRIE: inventory + catalog search
================================================================
  Your items starting with "" (name, quantity):
    health_potion    2
    torch            2
  Game items starting with "dr" (name, max stack):
    dragonbane       1
    dragon_scale     10
```

```
================================================================
  STEP 4 - GRAPH: quest dependencies
================================================================
  Quest graph: 6 quests, 7 edges, no cycles
  A valid order to play (topological sort):
    1. Wake in Riverwood      [done  ] needs: -
    2. Clear the Goblin Cave  [done  ] needs: Wake in Riverwood
    3. Find the Old Sword     [done  ] needs: Wake in Riverwood
    4. Forge Dragonbane       [done  ] needs: Find the Old Sword
    5. Consult the Oracle     [done  ] needs: Clear the Goblin Cave, Find the Old Sword
    6. Slay the Dragon        [active] needs: Forge Dragonbane, Consult the Oracle

================================================================
  STEP 5 - CORRUPT THE SAVE FILE, then LOAD GAME
================================================================
  Corrupting save.sav with damaged bytes
    bytes 135-137 damaged
  save.sav is CORRUPTED:
      [X] checksum mismatch (bytes were damaged)
  -> the save file is unusable; checkpoints will be used instead.

================================================================
  STEP 6 - CORRUPT THREE CHECKPOINTS
================================================================
  Corrupting checkpoint t=62 "Found the sword" with quest done out of order
    'Slay the Dragon' marked done before its prerequisites
  Corrupting checkpoint t=82 "Cleared the cave" with item glitch
    'health_potion' duplicated to x9999
    fake item 'debug_sword_of_doom' added
  Corrupting checkpoint t=118 "Forged Dragonbane" with damaged bytes
    bytes 58-60 damaged
```

```
================================================================
  STEP 7 - DETECT CORRUPTION
================================================================
  Checkpoints:
    t=37    Left the village         ok
    t=62    Found the sword          CORRUPT
      [X] 'Slay the Dragon' is done but 'Forge Dragonbane' is not done
    t=82    Cleared the cave         CORRUPT
      [X] health_potion x9999 is outside 1-10 (duplication glitch?)
      [X] unknown item 'debug_sword_of_doom'
    t=102   Met the oracle           ok
    t=103   Hurt by a monster        ok
    t=118   Forged Dragonbane        CORRUPT
      [X] checksum mismatch (bytes were damaged)
      [X] health_potion x253 is outside 1-10 (duplication glitch?)
  -> 3 of 6 checkpoints corrupted

  Live game: ok

  Save file: CORRUPT
      [X] checksum mismatch (bytes were damaged)

================================================================
  STEP 8 - COMPARE SAVES
================================================================
  Comparing t=37 with t=102:
    field               | first        | second
    --------------------+--------------+--------------
    hp                  | 88           | 58
    gold                | 133          | 359
    item types          | 1            | 2
```

```
    quests done          | 1           | 4
    quest Find the Old S | active      | done
    quest Clear the Gobl | active      | done
    quest Consult the Or | locked      | done
    quest Forge Dragonba | locked      | active
    integrity            | ok          | ok

  Comparing t=82 with t=102:
    field                | first       | second
    ---------------------+-------------+-------------
    hp                   | 71          | 58
    gold                 | 260         | 359
    item types           | 3           | 2
    quests done          | 3           | 4
    quest Consult the Or | active      | done
    integrity            | CORRUPT     | ok

================================================================
  STEP 9 - ROLLBACK to the latest valid checkpoint
================================================================
  skip t=118 "Forged Dragonbane": corrupted
  Rolled back to t=103 "Hurt by a monster"
    Aria | HP 10/100 | Gold 359
    Inventory: health_potion x2, torch x2
    Quests done: [Wake in Riverwood] [Find the Old Sword] [Clear the Goblin Cave] [Consult the Oracle]
  (Rollback takes the most RECENT valid state - even if it is a bad spot.)

================================================================
  STEP 10 - RECOVER BEST STATE (max-heap)
================================================================
  3 of 6 checkpoints are valid and go into the max-heap
  Ranking (score = 1000*quests + gold + 20*items, -1000 if HP is below 25):
```

```
  STEP 10 - RECOVER BEST STATE (max-heap)
================================================================
  3 of 6 checkpoints are valid and go into the max-heap
  Ranking (score = 1000*quests + gold + 20*items, -1000 if HP is below 25):
   >#1  t=102   Met the oracle          score 4399
    #2  t=103   Hurt by a monster       score 3399
    #3  t=37    Left the village        score 1153
  Recovered best state: t=102 "Met the oracle"
    Aria | HP 58/100 | Gold 359
    Inventory: health_potion x2, torch x2
    Quests done: [Wake in Riverwood] [Find the Old Sword] [Clear the Goblin Cave] [Consult the Oracle]
  (The heap prefers t=102 over t=103: same progress, but not nearly dead.)

================================================================
  STEP 11 - SAVE GUARD: a corrupted live state is never saved
================================================================
    HP set to 300 (max is 100)
  SAVE REFUSED: the live state is corrupted - saving it would spread the damage.
      [X] HP 300 outside 0-100
  3 of 6 checkpoints are valid and go into the max-heap
  Ranking (score = 1000*quests + gold + 20*items, -1000 if HP is below 25):
   >#1  t=102   Met the oracle          score 4399
    #2  t=103   Hurt by a monster       score 3399
    #3  t=37    Left the village        score 1153
  Recovered best state: t=102 "Met the oracle"
    Aria | HP 58/100 | Gold 359
    Inventory: health_potion x2, torch x2
    Quests done: [Wake in Riverwood] [Find the Old Sword] [Clear the Goblin Cave] [Consult the Oracle]
  Game saved to save.sav (checksum 6C5309AF)

Demo complete.
aviral@aviral-Nitro-ANV15-51:~/Downloads/Game_Save_Recovery_Engine /Game Save Recovery Engine$
```

6.3 Discussion

- All tests passed and all objectives were met.

- The checksum caught damaged bytes, and the trie and graph caught impossible values.
- Both checks are needed, because each one catches problems the other misses.
- Recovery gives a better result than rollback because it picks the best checkpoint, not just the latest.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

We built a working system in C that saves game progress, finds corrupted saves and recovers the best valid one. Each data structure has a clear job: the AVL tree for checkpoints, the trie for items, the graph for quests and the max-heap for recovery.

7.2 Limitations

- Checkpoints are lost when the program closes.
- The game is very small (8 items, 6 quests).
- The checksum is simple and not secure against a skilled attacker.
- Text menu only.

7.3 Future Scope

- Save checkpoints to disk.
- Use a stronger checksum.
- Add more items and quests.
- Add a graphical interface.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009.

[2] B. W. Kernighan and D. M. Ritchie, The C Programming Language, 2nd ed., Prentice Hall, 1988.

[3] GeeksforGeeks, "Insertion in an AVL Tree." https://www.geeksforgeeks.org/insertion-in-an-avl-tree/

[4] GeeksforGeeks, "Trie | (Insert and Search)." https://www.geeksforgeeks.org/trie-insert-and-search/

[5] GeeksforGeeks, "Topological Sorting using Kahn's Algorithm." https://www.geeksforgeeks.org/topological-sorting-indegree-based-solution/

[6] GeeksforGeeks, "Detect Cycle in a Directed Graph." https://www.geeksforgeeks.org/detect-cycle-in-a-graph/

[7] GeeksforGeeks, "Heap Data Structure." https://www.geeksforgeeks.org/heap-data-structure/

[8] Wikipedia, "Checksum." https://en.wikipedia.org/wiki/Checksum

[9] cppreference.com, "C Standard Library Reference." https://en.cppreference.com/w/c

APPENDIX A: ADDITIONAL MATERIAL

Source files: game.h, checkpoints.c, inventory.c, game.c, recovery.c (all in one folder).

Run ./game_recovery for the menu, or ./game_recovery --demo for the demo.